

Entregables de Sebastián — Efecto Mpemba cuántico

ítems 3, 4, 5 y 8: evolución temporal (a/b), trayectorias y relevancia macro

Sebastián

3 de junio de 2026

Documento de revisión. Cada sección se \input en el maestro mpemba_cuantico.tex.

Índice

1	Evolución temporal (a): integración directa con RK4	1
1.1	Explicación del framework	1
1.2	Del modelo teórico a la discretización (paso a paso)	1
1.3	Implementación serial y líneas de código	2
1.4	La directiva de paralelización (OpenMP)	2
1.5	Justificación de la paralelización: OpenMP	2
1.6	Comparación serial vs paralelo	3
1.7	Resultados y validación	3
2	Evolución temporal (b): acción del exponencial por Krylov	5
2.1	Explicación del framework	5
2.2	Del modelo teórico a la discretización (paso a paso)	5
2.3	Implementación serial y líneas de código	6
2.4	La directiva de paralelización (OpenMP)	6
2.5	Justificación de la paralelización: OpenMP	6
2.6	Comparación serial vs paralelo	7
2.7	Comparación de método (a vs b): el resultado central	7
2.8	Resultados y conclusión del método (b)	8
3	Muchos cuerpos: trayectorias cuánticas con MPI	8
3.1	Explicación del framework	8
3.2	Del modelo teórico a la discretización (paso a paso)	9
3.3	Implementación serial y líneas de código	10
3.4	Las llamadas de paralelización (MPI)	10
3.5	Justificación de la paralelización: MPI	10
3.6	Comparación serial vs paralelo	10
3.7	Resultados y validación	11
4	Influencia y relevancia en el efecto macroscópico clásico	12
4.1	La raíz macroscópica y su problema histórico	12
4.2	El mecanismo unificador: geometría espectral de la relajación	12
4.3	Lo que el enfoque cuántico devuelve al problema clásico	12
4.4	La firma universal: del macro clásico al cuántico	13
4.5	Relevancia	13

1 Evolución temporal (a): integración directa con RK4

1.1 Explicación del framework

La tarea de *evolución temporal* consiste en obtener la trayectoria del operador densidad $\rho(t)$ que resuelve la ecuación maestra de Lindblad $\dot{\rho} = \mathcal{L}[\rho]$ (ec. (??)) a partir de una preparación ρ_0 , para trazar las curvas de distancia al equilibrio $\mathcal{D}(\rho_t || \rho_{ss})$ que revelan (o descartan) el efecto Mpemba. El método (a) es la vía **directa**: integrar la ecuación diferencial paso a paso, sin construir el superoperador $\mathbb{L}$ de dimensión $d^2 \times d^2$, aplicando en su lugar la acción *matrix-free* $\mathcal{L}[\rho] = -i[H, \rho] + \sum_{\mu} (L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} \{L_{\mu}^{\dagger} L_{\mu}, \rho\})$.

1.2 Del modelo teórico a la discretización (paso a paso)

Paso 0 — Modelo teórico. El punto de partida es la ecuación maestra GKSL (??) para la cadena de Ising transversa disipativa:

$$\dot{\rho} = \mathcal{L}[\rho] = -i[H, \rho] + \sum_{\mu} \left(L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} \{L_{\mu}^{\dagger} L_{\mu}, \rho\} \right), \quad H = -J \sum_i Z_i Z_{i+1} - h \sum_i X_i,$$

con canales locales de bajada/subida $L_{i,-} = \sqrt{\gamma(\bar{n}+1)} \sigma_i^{-}$, $L_{i,+} = \sqrt{\gamma\bar{n}} \sigma_i^{+}$ y $\bar{n} = \bar{n}(2h, T)$ la ocupación de Bose del baño. (H y $\{L_{\mu}\}$ los construye `build_ising`.)

Paso 1 — Problema de valor inicial. Fijada una preparación ρ_0 (un estado de Gibbs a T_0), la evolución es el PVI $\dot{\rho} = \mathcal{L}[\rho]$, $\rho(0) = \rho_0$, planteado *en el espacio de operadores*. No hay malla espacial que discretizar: la “malla” es la propia base de Hilbert ($d = 2^N$ y ρ es $d \times d$). Lo que se discretiza es el **tiempo**.

Paso 2 — Semidiscretización temporal (RK4). Se parte $[0, t_{\max}]$ en pasos Δt con $\rho_n \approx \rho(n\Delta t)$ y se aplica el **Runge–Kutta clásico de 4.º orden** (explícito, un paso):

$$\begin{aligned} k_1 &= \mathcal{L}[\rho_n], & k_2 &= \mathcal{L}[\rho_n + \frac{\Delta t}{2} k_1], & k_3 &= \mathcal{L}[\rho_n + \frac{\Delta t}{2} k_2], \\ k_4 &= \mathcal{L}[\rho_n + \Delta t k_3], & \rho_{n+1} &= \rho_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

Error local $\mathcal{O}(\Delta t^5)$, global $\mathcal{O}(\Delta t^4)$.

Paso 3 — La operación discreta fundamental ($\mathcal{L}$ *matrix-free*). Cada k_i es una evaluación de $\mathcal{L}[\cdot]$. El punto clave de HPC es que **no** se forma el superoperador $\mathbb{L}$ de tamaño $d^2 \times d^2$; se evalúa la acción directamente sobre la matriz ρ con productos $d \times d$:

$$\mathcal{L}[V] = -i(HV - VH) + \sum_{\mu} \left(L_{\mu} V L_{\mu}^{\dagger} - \frac{1}{2} (L_{\mu}^{\dagger} L_{\mu} V + V L_{\mu}^{\dagger} L_{\mu}) \right).$$

Para la cadena de Ising son $2 + 3N_J$ productos de matrices densas $d \times d$ ($N_J = 2N$ canales), de coste $\mathcal{O}(d^3)$ cada uno; el coste por paso es $\mathcal{O}(N d^3)$. *Este producto de matrices es el hotspot que se paraleliza* (Listado 2).

1.3 Implementación serial y líneas de código

El núcleo está en `common/lindblad.hpp` (acción `apply_lindblad`, modelo `build_ising`, Gibbs por tiempo imaginario) y el integrador en `a_integracion_directa/src/rk4_evolution.cpp`. El Paso 2 (esquema RK4) es literalmente la función `rk4_step`:

```

1 static Mat rk4_step(const Mat& H, const std::vector<Mat>& Ls, const std::vector<Mat>& Lds,
2                    const std::vector<Mat>& LdL, const Mat& rho, double dt, int d) {
3     Mat k1 = apply_lindblad(H, Ls, Lds, LdL, rho, d);
4     Mat t2 = rho; axpy(t2, k1, 0.5*dt); Mat k2 = apply_lindblad(H, Ls, Lds, LdL, t2, d);
5     Mat t3 = rho; axpy(t3, k2, 0.5*dt); Mat k3 = apply_lindblad(H, Ls, Lds, LdL, t3, d);
6     Mat t4 = rho; axpy(t4, k3, dt);      Mat k4 = apply_lindblad(H, Ls, Lds, LdL, t4, d);
7     Mat out = rho;
8     for (int i = 0; i < d*d; ++i) out[i] += dt*(k1[i] + 2.0*k2[i] + 2.0*k3[i] + k4[i])
9         /6.0;
10    return out;
}

```

Listing 1: Paso RK4 sobre la ecuación maestra (rk4_evolution.cpp). Las cuatro etapas $k_1 \dots k_4$ son cuatro llamadas a la acción matrix-free `apply_lindblad` (Paso 3).

1.4 La directiva de paralelización (OpenMP)

La preparación inicial es un Gibbs a $T_0 = 5$ y la referencia el Gibbs del baño a $T = 0,8$. El *hotspot* absoluto es el producto de matrices densas `matmul`, y ahí —y solo ahí— está la directiva OpenMP: una única `#pragma omp parallel for` reparte las filas i del producto entre los hilos (cada fila es independiente, sin condiciones de carrera):

```

1 inline Mat matmul(const Mat& A, const Mat& B, int d) {
2     Mat C(d * d, cd(0, 0));
3     #pragma omp parallel for schedule(static)    // <-- DIRECTIVA: reparte las filas i
4         entre hilos
5     for (int i = 0; i < d; ++i) {
6         for (int k = 0; k < d; ++k) {
7             cd a = A[IDX(i, k, d)];
8             if (a == cd(0, 0)) continue;
9             const cd* brow = &B[IDX(k, 0, d)];
10            cd* crow = &C[IDX(i, 0, d)];
11            for (int j = 0; j < d; ++j) crow[j] += a * brow[j];
12        }
13    }
14    return C;
}

```

Listing 2: Producto de matrices con la directiva OpenMP (`common/lindblad.hpp`). Es el único punto paralelizado: el mismo binario, lanzado con distinto `OMP_NUM_THREADS`, da las curvas serie vs paralelo.

1.5 Justificación de la paralelización: OpenMP

El patrón de cómputo de la integración directa es: *miles de pasos secuenciales* (dependencia temporal estricta: ρ_{n+1} necesita ρ_n), cada uno dominado por productos de matrices densas que reutilizan los mismos datos. La dependencia temporal **impide** paralelizar entre pasos; el paralelismo disponible está *dentro* de cada paso, en el `matmul`. Ese es un patrón de **memoria compartida** con datos regulares y de tamaño moderado ($d \times d$ cabe en caché/RAM de un nodo): el paradigma natural es **OpenMP**, paralelizando el bucle de filas del `matmul` (`#pragma omp parallel for`). MPI sería contraproducente aquí (distribuir matrices pequeñas introduce comunicación de halo en cada uno de los miles de pasos); CUDA daría más ancho de banda pero no hay GPU disponible en la máquina de prueba (se discute como trabajo futuro en §2 y en la tabla ??). La corrección de la versión paralela se verifica de forma exacta: serial y multinúcleo producen el *mismo* estado final (coincidencia a precisión de máquina del observable D_{HS} final).

1.6 Comparación serial vs paralelo

El speedup es **sublineal** y satura cerca del número de núcleos físicos: se alcanza $S_{\max} \approx 2,9$ con 8 hilos, con eficiencia del 36 % a 8 hilos. El ajuste a la **ley de Amdahl** $S(p) = 1/[f + (1 - f)/p]$ da

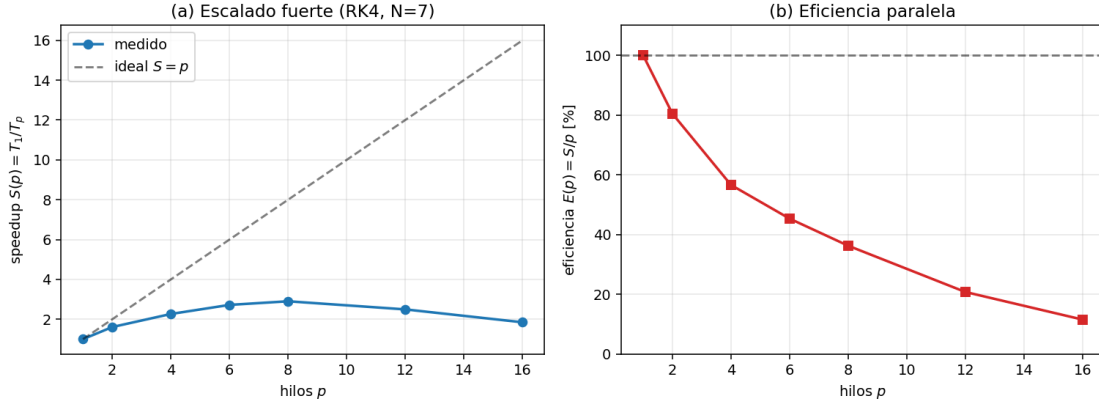


Figura 1: Escalado fuerte del integrador RK4 (cadena de Ising disipativa $N = 7$, $d = 128$) en un AMD Ryzen 7 7730U (8 núcleos físicos / 16 hilos). (a) speedup $S(p) = T_1/T_p$ frente al ideal $S = p$; (b) eficiencia paralela $E(p) = S/p$.

una fracción serie efectiva $f \approx 0,25$. Las causas de la sublinealidad son típicas de este patrón: (i) el integrador hace *fork/join* de OpenMP en cada uno de los muchos `matmul` pequeños, con sobrecoste acumulado; (ii) el problema es *memory-bandwidth bound* (las matrices se releen de memoria), de modo que los hilos compiten por el ancho de banda compartido; (iii) las asignaciones de memoria temporales y las operaciones vectoriales fuera del `matmul` son seriales. El salto de 8 a 16 hilos aporta poco porque los 16 hilos son *hyperthreads* sobre 8 núcleos físicos, sin ancho de banda adicional.

Para ver *directamente* la mejora —y no solo el speedup adimensional— la Figura 2 enfrenta el **tiempo de cómputo en serie (1 hilo) y en paralelo (8 hilos)** en función del tamaño del sistema N (la malla del operador, $d = 2^N$), a igual número de pasos. Ambas curvas crecen como $\sim d^3$, pero la paralela queda sistemáticamente por debajo: la *separación* entre las dos es la ganancia, y crece con la malla porque a mayor d el `matmul` domina y amortiza mejor el coste de *fork/join*.

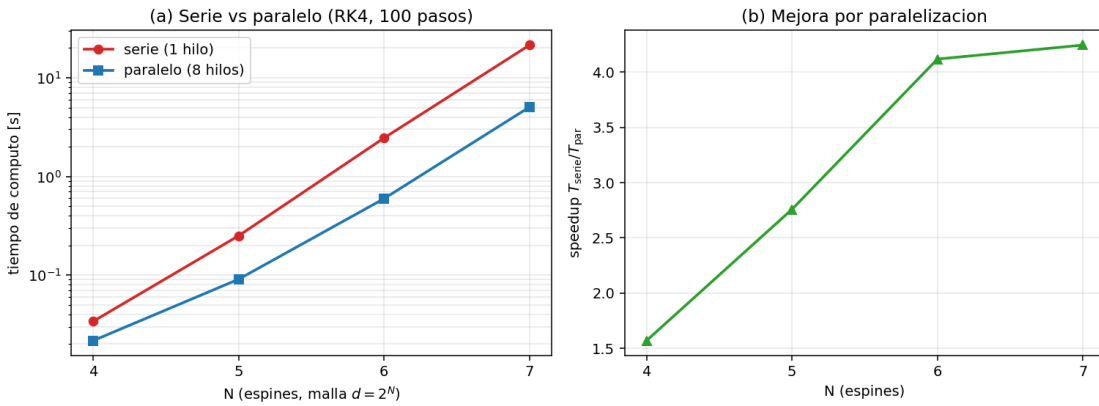


Figura 2: Tiempo de cómputo **serie vs paralelo** del RK4 frente al tamaño del sistema N (malla $d = 2^N$), a 100 pasos fijos. (a) las dos curvas (1 hilo en rojo, 8 hilos en azul, escala log) muestran el mismo coste $\sim d^3$ desplazado: el hueco entre ellas es la mejora. (b) el speedup $T_{\text{serie}}/T_{\text{par}}$ resultante crece con la malla (de $\sim 1,6$ en $N = 4$ a ~ 4 en $N = 7$): cuanto mayor es d , más domina el `matmul` y mejor se amortiza el *fork/join*, aunque siempre sublineal frente al ideal de 8.

1.7 Resultados y validación

Correctitud y orden del método. La figura 3 valida el integrador frente a la evolución espectral *exacta* $e^{t\mathcal{L}}\rho_0$ (suma de modos): el error a tiempo fijo decae como Δt^4 , confirmando el **orden 4** del RK4 (pendiente medida $\approx 4,04$).

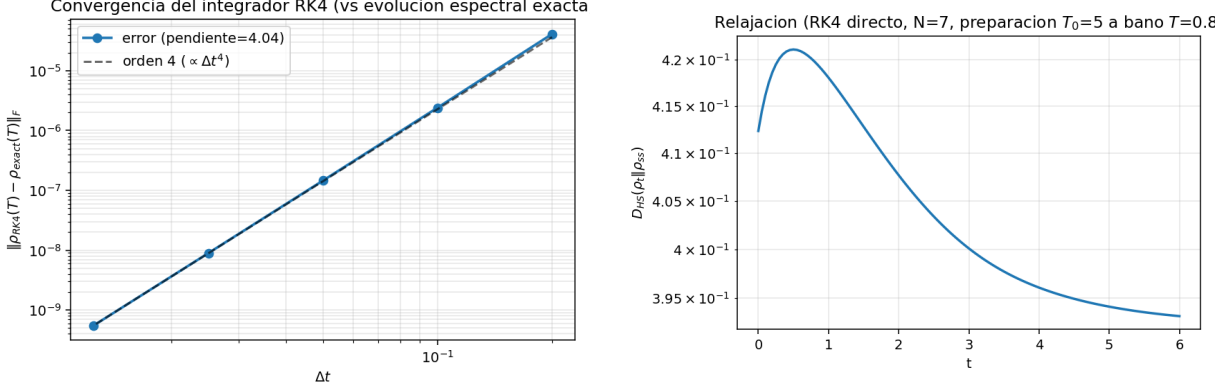


Figura 3: Izquierda: convergencia de orden 4 del RK4 frente a la solución exacta. Derecha: curva de relajación $D_{HS}(t)$ producida por el método (preparación caliente $T_0 = 5$ relajando al baño $T = 0,8$).

Coste con el tamaño. La figura 4 confirma el escalado $\mathcal{O}(d^3)$ por paso esperado del producto de matrices densas: al pasar de N a $N + 1$ ($d \rightarrow 2d$) el coste por paso se multiplica por ~ 8 . Es justamente este crecimiento el que vuelve inviable la representación densa de ρ para N grande y motiva las trayectorias (§3) en el régimen de muchos cuerpos.

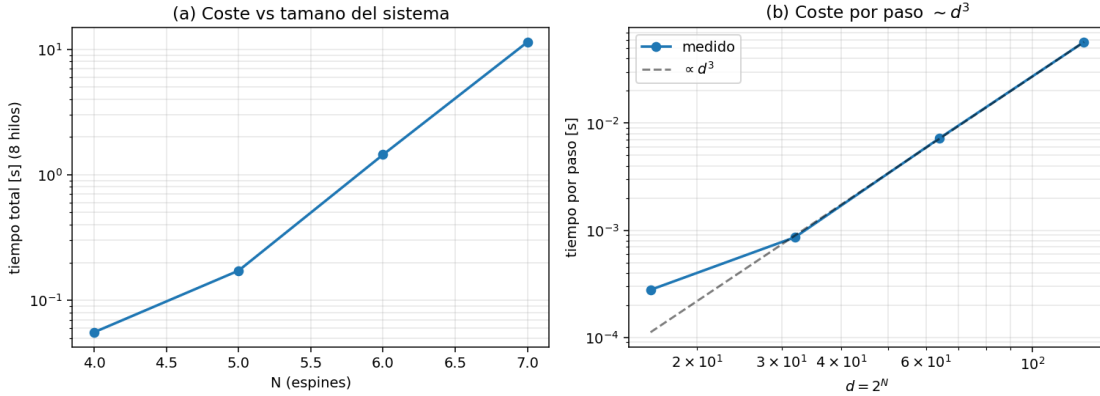


Figura 4: Coste del método con el tamaño del sistema (8 hilos). (a) tiempo total; (b) tiempo por paso frente a la referencia $\propto d^3$.

Conclusión del método (a). La integración directa por RK4 es *simple*, *exacta a orden 4* y *robusta*, y se paraleliza de forma natural con OpenMP a nivel del producto de matrices. Su limitación es doble: el coste $\mathcal{O}(d^3)$ por paso y la necesidad de pasos pequeños por estabilidad/precisión, que obligan a muchas evaluaciones del Liouvilliano. El método (b) (acción del exponencial, §2) ataca precisamente este segundo punto.

2 Evolución temporal (b): acción del exponencial por Krylov

2.1 Explicación del framework

El método (b) resuelve la misma evolución $\dot{\rho} = \mathcal{L}[\rho]$ pero por una vía distinta: en lugar de integrar paso a paso, aproxima directamente la **acción del exponencial del Liouvilliano**,

$$\rho(t + \tau) = e^{\tau\mathcal{L}}[\rho(t)],$$

que es la solución *exacta* en un paso de tamaño τ . La dificultad es que $e^{\tau\mathcal{L}}$ es un objeto de dimensión $d^2 \times d^2$ imposible de formar. La idea de Krylov es proyectar la acción sobre el subespacio $\mathcal{K}_m = \text{span}\{\rho, \mathcal{L}\rho, \dots, \mathcal{L}^{m-1}\rho\}$, de dimensión $m \ll d^2$, donde el exponencial es trivial de calcular.

2.2 Del modelo teórico a la discretización (paso a paso)

Paso 0 — Modelo teórico. El mismo de (a): la ecuación maestra GKSL $\dot{\rho} = \mathcal{L}[\rho]$ para la cadena de Ising disipativa (Hamiltoniano H y canales L_μ idénticos, §1.2).

Paso 1 — Solución formal exacta. En lugar de integrar la EDO, se usa que la solución del PVI es *exacta* en un paso de tamaño τ mediante el exponencial del generador:

$$\rho(t + \tau) = e^{\tau\mathcal{L}}[\rho(t)].$$

El obstáculo: $e^{\tau\mathcal{L}}$ vive en el superoperador de dimensión $d^2 \times d^2$, imposible de formar y exponenciar para N grande.

Paso 2 — Discretización por proyección de Krylov (Arnoldi). Se proyecta la acción del exponencial sobre el subespacio de Krylov $\mathcal{K}_m = \text{span}\{\rho, \mathcal{L}\rho, \dots, \mathcal{L}^{m-1}\rho\}$ ($m \ll d^2$). Con **Arnoldi** se construye una base ortonormal $\{Q_0, \dots, Q_{m-1}\}$ (cada Q_i es un operador $d \times d$; producto interno de Frobenius $\langle A, B \rangle = \text{Tr}(A^\dagger B)$) y la matriz de Hessenberg $H_m \in \mathbb{C}^{m \times m}$ que representa $\mathcal{L}$ en esa base. El *único* ingrediente que toca el sistema grande es la acción $w = \mathcal{L}[Q_j]$ en cada iteración de Arnoldi.

Paso 3 — Exponencial pequeño y reconstrucción. La acción discretizada es

$$e^{\tau\mathcal{L}}[\rho] \approx \|\rho\|_F \sum_{i=0}^{m-1} [e^{\tau H_m}]_{i,0} Q_i,$$

con $e^{\tau H_m}$ calculado sobre la matriz *pequeña* $m \times m$ por *scaling-and-squaring* con serie de Taylor (`small_expm`). Coste por bloque: m aplicaciones de $\mathcal{L}$ (Arnoldi) $+ \mathcal{O}(m d^2)$ y un $e^{\tau H_m}$ de coste $\mathcal{O}(m^3)$ despreciable. La ventaja decisiva: para τ dentro del radio de convergencia, $m \sim 20\text{--}30$ basta para precisión de máquina y *un solo bloque avanza un paso de tiempo grande*.

2.3 Implementación serial y líneas de código

En `b_accion_exponencial/src/krylov_evolution.cpp`. El bucle de Arnoldi (Paso 2) es el corazón del método; nótese que cada iteración llama a la misma acción `apply_lindblad` de (a) —es decir, *comparte el hotspot paralelizable*—:

```
1 for (int j = 0; j < m; ++j) {
2     Mat w = apply_lindblad(H, Ls, Lds, LdL, Q[j], d); ++L_applies;    // <-- hotspot (
3                                     // ortogonalizacion de Gram-
4                                     // Schmidt
5     double hij = fro_dot(Q[i], w); Hess[i*m+j] = hij; axpy(w, Q[i], -hij);
6     double hj = fro_norm(w);
```

```

7   Hess[(j+1)*m+j] = hj;
8   if (hj < 1e-12) { mm = j + 1; break; } // convergencia anticipada
9   Q.push_back(scaled(w, 1.0 / hj));
10 }

```

Listing 3: Bucle de Arnoldi de `krylov_step` (`krylov_evolution.cpp`): la línea `apply_lindblad(...)` es la acción matrix-free de $\mathcal{L}$, donde se cuenta el trabajo (`L_applies`) y donde reside la paralelización OpenMP (vía `matmul`, Listado 4).

2.4 La directiva de paralelización (OpenMP)

El *hotspot* es el mismo que en (a): el producto de matrices densas, llamado m veces por bloque dentro de `apply_lindblad`. La directiva es por tanto idéntica —una sola `#pragma omp parallel for` en `matmul` (`common/lindblad.hpp`)—, lo que explica que el escalado paralelo de (b) sea equivalente al de (a):

```

1   #pragma omp parallel for schedule(static) // <-- DIRECTIVA: filas i repartidas entre
    hilos
2   for (int i = 0; i < d; ++i)
3   for (int k = 0; k < d; ++k) { /* C[i,:] += A[i,k] * B[k,:] */ }

```

Listing 4: La misma directiva OpenMP que en T2(a): el bucle de filas de `matmul` es el único punto paralelizado (compartido por RK4 y Krylov).

2.5 Justificación de la paralelización: OpenMP

Como en el método (a), el *hotspot* es la acción $\mathcal{L}[\cdot]$ (el producto de matrices densas), llamada m veces por bloque. La ortogonalización de Arnoldi añade productos internos de Frobenius de coste $\mathcal{O}(d^2)$, mucho menores que el $\mathcal{O}(d^3)$ del `matmul`. Por tanto el patrón sigue siendo de **memoria compartida** dominado por `matmul`, y el paradigma natural vuelve a ser **OpenMP** sobre el mismo bucle. (En el régimen de muchos cuerpos, donde d^2 no cabe en memoria, la acción de Krylov se paraleliza con MPI distribuyendo el vector $|\rho\rangle$; aquí, con ρ denso en un nodo, OpenMP es lo adecuado.) La corrección se verifica de dos formas: serial y paralelo dan el mismo resultado, y el resultado coincide con el del método (a) (RK4) a todas las cifras: $D_{HS}^{\text{final}} = 0,4241128226$ para $N = 6$, idéntico por ambos métodos (referencia exacta por evolución espectral: $D_{HS}^{\text{exacto}} = 0,42411283$).

2.6 Comparación serial vs paralelo

La Figura 6 muestra la mejora de forma directa: el **tiempo de cómputo en serie (1 hilo) frente al paralelo (8 hilos)** en función del tamaño del sistema N (malla $d = 2^N$), a igual paso τ y misma dimensión de Krylov m . Igual que en (a), las dos curvas crecen como $\sim d^3$ y el hueco entre ellas —la ganancia— se ensancha con la malla.

2.7 Comparación de método (a vs b): el resultado central

La ventaja de Krylov no está en el escalado paralelo (idéntico al de RK4) sino en el **trabajo necesario para una precisión dada**. La figura 7 mide el error del observable D_{HS} final frente a una referencia de alta precisión, en función del número de aplicaciones de $\mathcal{L}$ (la operación cara, independiente de la máquina):

La interpretación física/numérica: RK4 converge *algebraicamente* (el error baja como Δt^4), mientras que Krylov converge *super-algebraicamente* (casi exponencial en m) porque captura la acción exacta del exponencial dentro del subespacio. En este sistema disipativo *no rígido*, ambos métodos son de trabajo comparable para precisiones moderadas, pero Krylov presenta tres ventajas estructurales: (i) a igual trabajo es $\sim 25\times$ más preciso; (ii) usa pasos de tiempo $5\times$ mayores ($\tau = 1,0$ frente a $\Delta t = 0,2$) manteniéndose estable y exacto; (iii) es **incondicionalmente estable**,

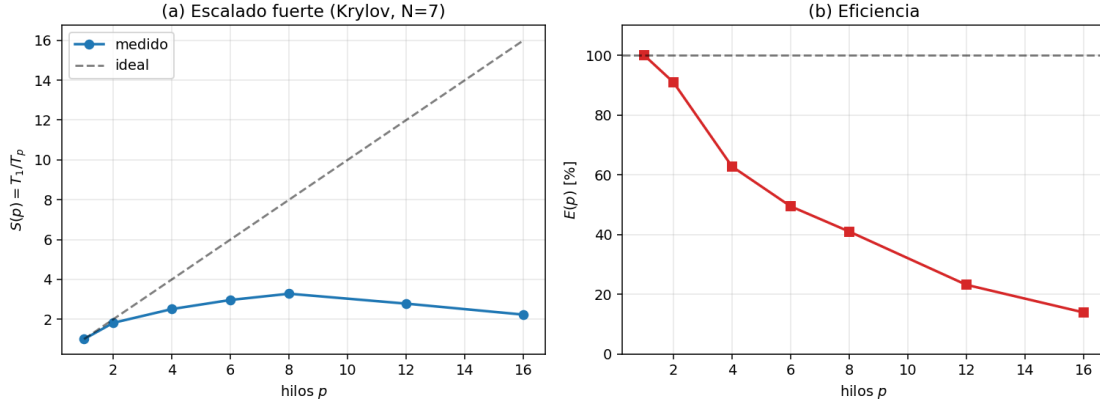


Figura 5: Escalado fuerte del método de Krylov (Ising disipativo $N = 7$, $d = 128$). El comportamiento es análogo al del método (a): speedup sublineal que satura en los núcleos físicos, por las mismas causas (fork/join del `matmul`, *memory-bandwidth bound*, hyperthreading sin ancho de banda extra). Pico $S_{\max} \approx 3,3$, eficiencia 41 % a 8 hilos (ligeramente mejor que RK4: hay menos llamadas a `matmul` por unidad de trabajo).

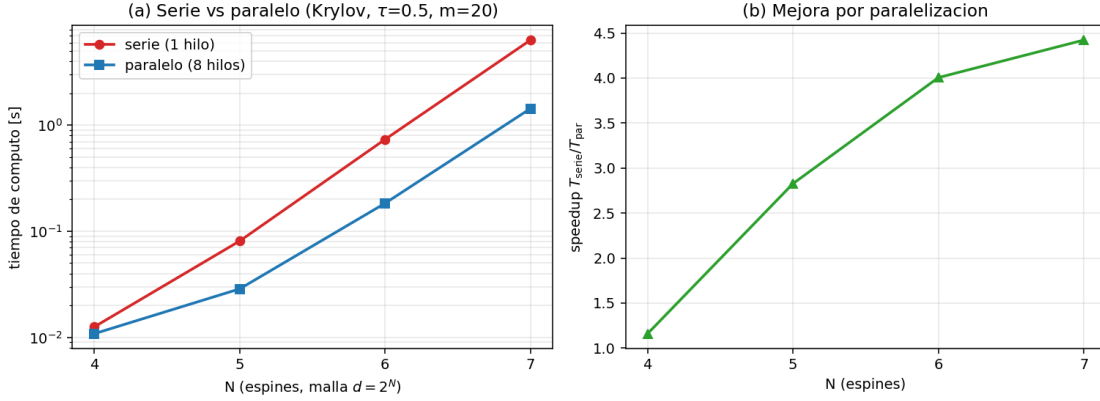


Figura 6: Tiempo de cómputo **serie vs paralelo** del método de Krylov frente al tamaño del sistema N (malla $d = 2^N$), a $\tau = 0,5$ y $m = 20$ fijos. (a) curvas de 1 hilo (rojo) y 8 hilos (azul) en escala log; (b) speedup $T_{\text{serie}}/T_{\text{par}}$ resultante. El patrón es el mismo que el de RK4 porque el hotspot paralelo —el `matmul` de la acción de $\mathcal{L}$ — es común a ambos métodos.

mientras que RK4 tiene un límite de estabilidad $\Delta t < 2,8/|\lambda_{\max}|$. Esta ventaja se vuelve *decisiva* para generadores *rígidos* (escalas de energía grandes o disipación fuerte) y para alcanzar el estado estacionario, donde el límite de estabilidad obliga a RK4 a dar muchísimos pasos diminutos. RK4 sigue siendo preferible cuando se necesita la trayectoria *densamente* muestreada en el tiempo o cuando el Liouvilliano depende del tiempo.

2.8 Resultados y conclusión del método (b)

La figura 8 muestra la curva de relajación obtenida con pasos *grandes* ($\tau = 0,25$), inalcanzables para RK4 a esa precisión. Resumen de la comparación a–b:

- Misma física (resultados idénticos), misma paralelización (OpenMP sobre `matmul`, escalado sublineal equivalente).
- Krylov (b): mucho menos trabajo para alta precisión y pasos de tiempo grandes; ideal para llegar al estacionario o muestrear pocos tiempos.

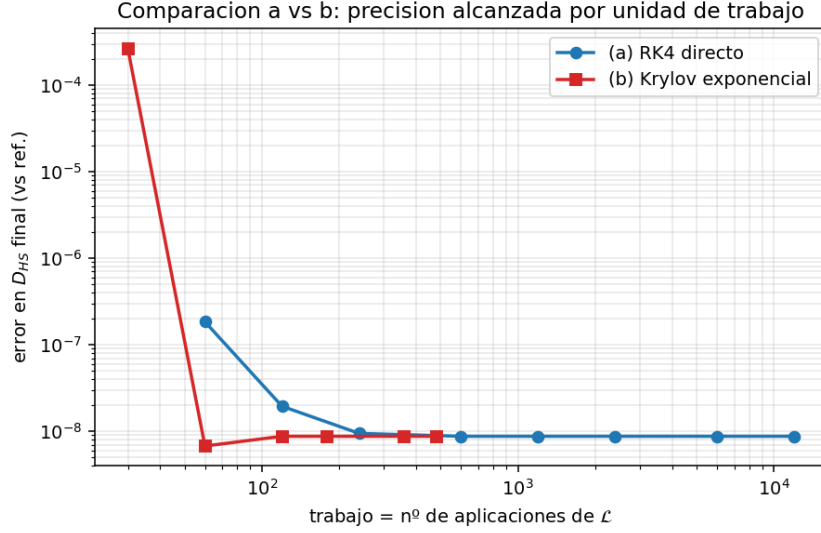


Figura 7: Error del observable D_{HS} final frente a la referencia exacta (evolución espectral), en función del trabajo. **A igual trabajo (≈ 60 aplicaciones de $\mathcal{L}$), Krylov es $\sim 25\times$ más preciso** (7×10^{-9} frente a 2×10^{-7} de RK4). El punto Krylov de $m = 10$ (no convergido, 2×10^{-4}) cae cinco órdenes de magnitud al pasar a $m = 20$: convergencia *super-algebraica* en m , frente a la algebraica ($\propto \Delta t^4$) de RK4. El suelo $\sim 9 \times 10^{-9}$ es la precisión de la propia referencia.

- RK4 (a): más simple, robusto, y mejor para trayectorias densas o generadores dependientes del tiempo.

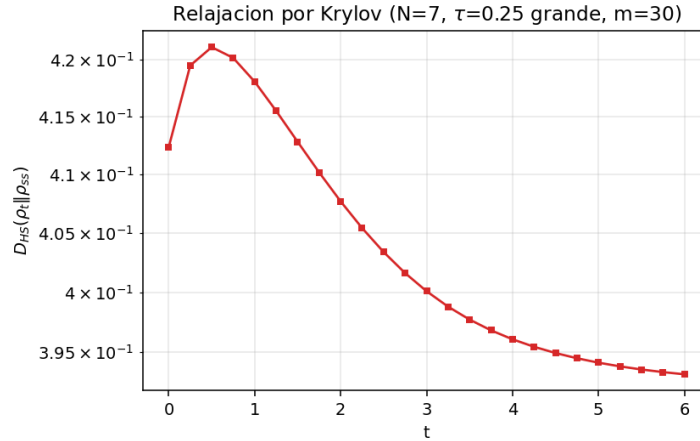


Figura 8: Relajación $D_{HS}(t)$ por Krylov con pasos grandes $\tau = 0,25$ ($N = 7$).

3 Muchos cuerpos: trayectorias cuánticas con MPI

3.1 Explicación del framework

En el régimen de *muchos cuerpos* la representación densa del operador densidad ρ se vuelve inviable: para N qubits, ρ tiene $d^2 = 4^N$ elementos y el Liouvilliano $4^N \times 4^N$. El método de **trayectorias cuánticas** (*Monte Carlo wavefunction*, MCWF) sorteja este muro: en lugar de propagar ρ , propaga M *vectores de estado puros* $|\psi\rangle \in \mathbb{C}^d$ y recupera el operador densidad como promedio

estocástico

$$\rho(t) = \mathbb{E}[|\psi(t)\rangle\langle\psi(t)|] \approx \frac{1}{M} \sum_{j=1}^M |\psi_j(t)\rangle\langle\psi_j(t)|.$$

Esto reduce la memoria de d^2 a d y, sobre todo, convierte el problema en uno **embarrassingly parallel**: cada trayectoria es independiente.

3.2 Del modelo teórico a la discretización (paso a paso)

Paso 0 — Modelo teórico. El mismo generador GKSL $\dot{\rho} = \mathcal{L}[\rho]$ de la cadena de Ising disipativa (§1.2).

Paso 1 — Unraveling estocástico. En vez de propagar ρ , se reescribe la ecuación maestra como un promedio sobre *trayectorias* de estados puros (*Monte Carlo wavefunction*, Dalibard–Mølmer–Castin): $\rho(t) = \mathbb{E}[|\psi(t)\rangle\langle\psi(t)|]$. La parte coherente más la parte “de no salto” del disipador se agrupan en un **Hamiltoniano efectivo no hermítico**

$$H_{\text{eff}} = H - \frac{i}{2} \sum_{\mu} L_{\mu}^{\dagger} L_{\mu},$$

cuya evolución $|\dot{\psi}\rangle = -iH_{\text{eff}}|\psi\rangle$ *no* conserva la norma; el defecto de norma es exactamente la probabilidad de salto.

Paso 2 — Discretización temporal de cada trayectoria. Con paso Δt , cada trayectoria alterna:

1. **Evolución determinista** $|\dot{\psi}\rangle = -iH_{\text{eff}}|\psi\rangle$ por **RK4** (4.^o orden; elimina el sesgo $\mathcal{O}(\Delta t)$ de un Euler ingenuo). Resulta ψ' con $\|\psi'\|^2 = 1 - \delta p$.
2. **Salto estocástico.** Probabilidad $\delta p = 1 - \|\psi'\|^2$; se sortea $\varepsilon \sim U(0, 1)$: si $\varepsilon < \delta p$ hay *salto* (se elige el canal μ con probabilidad $\propto \|L_{\mu}\psi\|^2$ y $|\psi\rangle \leftarrow L_{\mu}|\psi\rangle / \|L_{\mu}\psi\|$); si no, $|\psi\rangle \leftarrow \psi' / \|\psi'\|$.

Paso 3 — Promedio de conjunto. El observable se recupera promediando las M trayectorias, $\rho(t) \approx \frac{1}{M} \sum_{j=1}^M |\psi_j(t)\rangle\langle\psi_j(t)|$. Coste por paso: unos pocos productos matriz–vector $\mathcal{O}(d^2)$ (no $\mathcal{O}(d^3)$); precisión del promedio $\propto M^{-1/2}$. *Las M trayectorias son independientes* $\Rightarrow$ el promedio es la única operación colectiva (Paso 3 = una MPI_Reduce).

3.3 Implementación serial y líneas de código

En T3/integration/src/trajectories_mpi.cpp (reutiliza T2/common/lindblad.hpp para el modelo y matvec). El estado inicial es el puro $|0\rangle$ y la referencia el Gibbs del baño. Cada trayectoria usa un generador mt19937_64 sembrado con su índice *global*, lo que hace el resultado **reproducible e independiente del número de rangos**. El Paso 2 (no-salto vs salto) es literalmente:

```

1 double dp = 1.0 - norm2; // probabilidad de salto
2 if (U(rng) < dp) { // --- SALTO ---
3     /* elegir canal mu con prob ~ ||L_mu psi||^2 ... */
4     std::vector<cd> Lp = matvec(Ls[mu], psi, d);
5     for (int a = 0; a < d; ++a) psi[a] = Lp[a] / nn; // psi <- L_mu psi / ||.||
6 } else { // --- NO salto: renormalizar ---
7     double inv = 1.0 / std::sqrt(norm2);
8     for (int a = 0; a < d; ++a) psip[a] = psi[a] * inv;
9 }

```

Listing 5: Decisión determinista/salto por trayectoria (trajectories_mpi.cpp). psip es el estado tras el paso RK4 de H_{eff} ; norm2 = $\|\psi'\|^2$.

3.4 Las llamadas de paralelización (MPI)

El paralelismo es de *tareas*: cada rango calcula su lote de trayectorias sin comunicación, y una sola reducción promedia. Las tres líneas MPI que estructuran todo el cálculo son el arranque, el reparto y la reducción final:

```

1 MPI_Init(&argc, &argv); // arranque del entorno MPI
2 MPI_Comm_rank(MPI_COMM_WORLD, &rank); // id de este rango
3 MPI_Comm_size(MPI_COMM_WORLD, &size); // numero total de rangos
4 // ... reparto balanceado de las M trayectorias (embarrassingly parallel):
5 int per = M / size, rem = M % size;
6 int my_M = per + (rank < rem ? 1 : 0); // cuantas calcula este rango
7 int my_start = rank * per + std::min(rank, rem);
8 // ... cada rango integra sus my_M trayectorias en un acumulador local 'acc' ...
9 // UNICA comunicacion: sumar los acumuladores locales -> rho(t) en el rango 0
10 MPI_Reduce(reinterpret_cast<double*>(acc.data()),
11            rank == 0 ? reinterpret_cast<double*>(glob.data()) : nullptr,
12            (int)(2 * acc.size()), MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

```

Listing 6: Esqueleto MPI de `trajectories_mpi.cpp`: arranque e identificación del rango, reparto balanceado de las M trayectorias entre rangos (sin comunicación), y la *única* operación colectiva `MPI_Reduce` que suma los acumuladores en el rango 0 (Paso 3, el promedio).

3.5 Justificación de la paralelización: MPI

El patrón de cómputo es radicalmente distinto al de la evolución temporal (T2): no hay una larga cadena de pasos dependientes sobre un único objeto grande, sino M cálculos *completamente independientes* sobre objetos pequeños ($|\psi\rangle \in \mathbb{C}^d$), seguidos de una *única reducción* (el promedio). Este es el caso de libro del paradigma **MPI** de paralelismo de tareas: cada rango calcula su lote de trayectorias sin comunicación alguna y al final una sola `MPI_Reduce` promedia los acumuladores. No hay comunicación de halo ni dependencias, por lo que el escalado es *casi ideal* y *multinodo* (a diferencia de OpenMP, limitado a un nodo y al ancho de banda compartido). CUDA sería un acelerador complementario *dentro* de cada rango (evolucionar cientos de trayectorias en lote en una GPU), pero el reparto grueso natural es MPI. La corrección se verifica de dos formas: (i) el resultado es idéntico con 1, 4 u 8 rangos (misma semilla por índice global $\Rightarrow$ reducción correcta); (ii) el promedio reproduce la ecuación maestra exacta (validación abajo).

3.6 Comparación serial vs paralelo

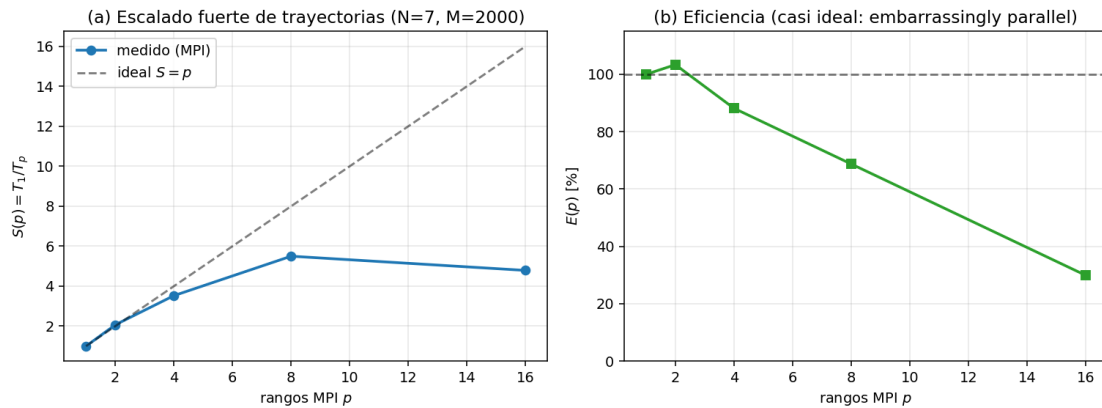


Figura 9: Escalado fuerte de las trayectorias cuánticas sobre rangos MPI ($N = 7$, $M = 2000$ fijo). El speedup es **casi ideal** ($S_{\max} \approx 5,5$ con 8 rangos, eficiencia 69%), en marcado contraste con el escalado sublineal del OpenMP de la evolución temporal (T2, $S \approx 2,9-3,3$). La razón: ausencia total de comunicación y dependencias entre trayectorias.

Contraste de paradigmas (el resultado clave del reparto T2 vs T3). La evolución temporal (T2) paraleliza *dentro* de un paso con OpenMP y satura pronto (memoria compartida, ancho de banda, fork/join): $S \lesssim 3,3$. Las trayectorias (T3) paralelizan *entre* tareas independientes con MPI y escalan casi linealmente: $S \approx 5,5$. Es la ilustración perfecta de que el paradigma óptimo lo dicta el *patrón de cómputo*, no una preferencia a priori.

La Figura 10 hace explícita la mejora variando el parámetro propio del método —el **número de trayectorias** M , que fija el trabajo Monte Carlo— y enfrentando el **tiempo en serie (1 rango)** y **en paralelo (8 rangos)**. Ambas crecen linealmente con M (coste $\propto M$), pero con pendientes muy distintas: las rectas *divergen*, y el cociente entre ellas se mantiene casi constante y cercano al número de núcleos, precisamente porque no hay comunicación entre trayectorias.

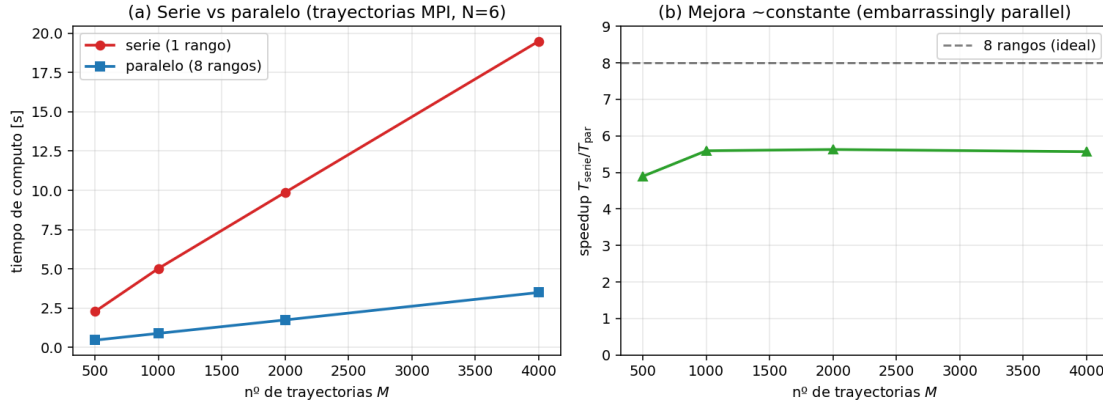


Figura 10: Tiempo de cómputo **serie vs paralelo** de las trayectorias frente al número de trayectorias M ($N = 6$, $d = 64$). (a) serie (1 rango, rojo) y paralelo (8 rangos, azul) crecen como $\propto M$ con pendientes distintas: el hueco —la mejora— se ensancha con M . (b) el speedup $T_{\text{serie}}/T_{\text{par}}$ es *casi constante* en M (línea de referencia ideal = 8), firma del paralelismo *embarrassingly parallel*.

3.7 Resultados y validación

Convergencia estadística. La figura 11 muestra que el error del observable D_{HS} final frente a la ecuación maestra exacta (evolución espectral desde $|0\rangle$, $D_{HS}^{\text{exacto}} = 0,40490$) decae como $M^{-1/2}$ (pendiente medida $\approx -0,6$, consistente con $M^{-1/2}$), confirmando la convergencia de Monte Carlo. El uso de RK4 en la parte determinista elimina el sesgo sistemático: a $\Delta t = 0,02$ y $M = 8000$ el resultado coincide con el exacto con una diferencia de $\sim 0,002$.

Conclusión del método de trayectorias. Las trayectorias cuánticas son el método de elección para muchos cuerpos: memoria $\mathcal{O}(d)$ por trayectoria, coste $\mathcal{O}(d^2)$ por paso y, sobre todo, **escalado MPI casi ideal** que aprovecha clústeres multinodo. El precio es la convergencia $M^{-1/2}$ (para una cifra más de precisión hacen falta $100\times$ trayectorias), mitigable precisamente por la abundancia de paralelismo. Para cadenas largas y observables de entrelazamiento, el relevo natural son las redes tensoriales (entregable de Juan, §??).

4 Influencia y relevancia en el efecto macroscópico clásico

4.1 La raíz macroscópica y su problema histórico

El efecto Mpemba nació como una observación *macroscópica y clásica*: agua inicialmente más caliente que se congela antes que agua más fría (Mpemba–Osborne 1969). Durante medio siglo fue un fenómeno disputado, porque a escala macroscópica intervienen muchos mecanismos difíciles de controlar (convección, evaporación, sobreenfriamiento, gases disueltos) y porque *carecía de un*

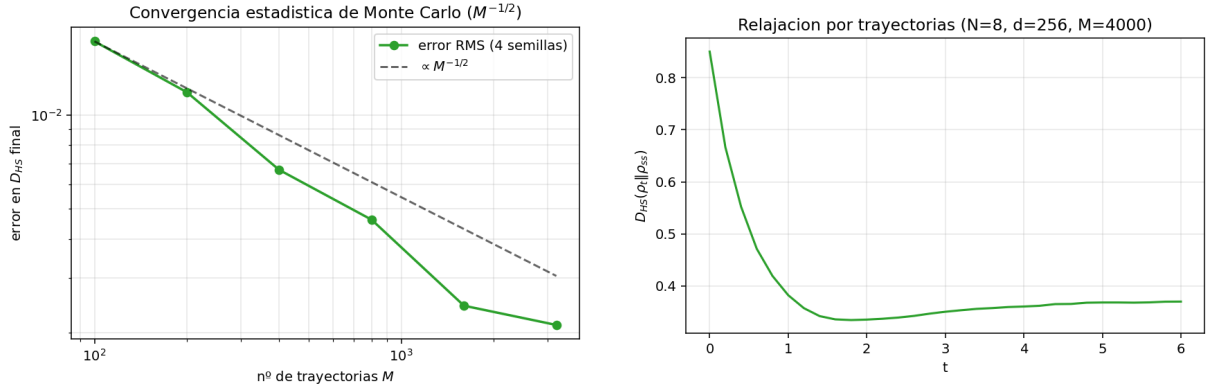


Figura 11: Izquierda: convergencia estadística $M^{-1/2}$ del promedio de trayectorias. Derecha: curva de relajación $D_{HS}(t)$ para $N = 8$ ($d = 256$, $M = 4000$), un sistema en el que el método de trayectorias usa vectores de 256 componentes en lugar de operadores densidad de 256^2 .

criterio teórico limpio: ¿qué significa exactamente que un sistema esté “más lejos del equilibrio”? Distintas métricas de distancia daban respuestas distintas (la crítica de Burrige–Linden 2016).

4.2 El mecanismo unificador: geometría espectral de la relajación

El marco cuántico desarrollado en este trabajo aporta, precisamente, lo que le faltaba al fenómeno macroscópico: un **mecanismo espectral riguroso** que es *el mismo* en el régimen clásico y en el cuántico. En ambos casos la relajación se descompone en modos,

$$\text{clásico: } \mathbf{p}(t) = \pi + \sum_{k \geq 2} a_k e^{\lambda_k t} \mathbf{v}_k, \quad \text{cuántico: } \rho(t) = \rho_{ss} + \sum_{k \geq 2} a_k e^{\lambda_k t} \hat{r}_k,$$

con $\mathbf{v}_k$ los autovectores de la matriz de tasas y $\hat{r}_k$ los autooperadores del Liouvillian. El efecto Mpemba ocurre, en *los dos* mundos, cuando la preparación caliente *solapa poco con el modo lento* ($a_2 \approx 0$): es un fenómeno de **geometría no-normal** de los modos de relajación, no una propiedad del agua. Lo que en el agua era una anomalía empírica resulta ser un caso particular de una estructura matemática general.

4.3 Lo que el enfoque cuántico devuelve al problema clásico

Tres aportaciones del marco cuántico iluminan directamente el efecto macroscópico:

1. **Criterio espectral del Mpemba fuerte** ($a_2 = 0$, Carollo *et al.*): da una condición *exacta* y constructiva para la aceleración, válida también para sistemas clásicos de muchos estados (vidrios, granulares).
2. **Diagnóstico independiente de la métrica** (termomayorización, Vu–Hayakawa): resuelve la ambigüedad histórica de Burrige–Linden —certifica el efecto bajo *todas* las distancias monótonas a la vez— y es aplicable tal cual a las distribuciones clásicas.
3. **Los puntos excepcionales y el recurso metrológico**: conceptos nacidos en el régimen cuántico que sugieren nuevos protocolos (aceleración por preparación, termometría fuera del equilibrio) trasladables a plataformas clásicas controladas.

4.4 La firma universal: del macro clásico al cuántico

La figura 12 muestra la *misma* firma —el cruce de las curvas de relajación, donde una preparación que parte más lejos alcanza el equilibrio antes— en tres dominios físicos usando los datos del

repositorio: un coloide clásico en trampa óptica (Kumar 2022), un circuito cuántico de muchos cuerpos (Turkeshi 2024) y la simulación cuántica propia de este trabajo. La continuidad conceptual es total: el mismo criterio de solapamiento espectral explica el cruce en los tres casos.

La firma universal del efecto Mpemba: cruce de curvas de relajacion, del macro clasico al cuantico

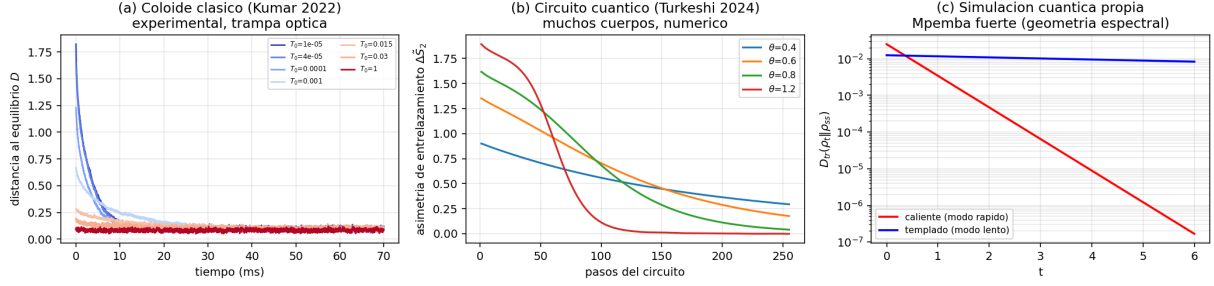


Figura 12: La firma universal del efecto Mpemba —el cruce de curvas de relajación— del macro clásico al cuántico. (a) coloide clásico experimental (Kumar 2022): la distancia al equilibrio de una preparación caliente cruza por debajo de otra más fría. (b) circuito cuántico de muchos cuerpos (Turkeshi 2024): la asimetría de entrelazamiento de $\theta = 1,2$ parte como la mayor pero se restaura antes. (c) simulación cuántica propia (Mpemba fuerte por geometría espectral). El mecanismo —solapamiento con los modos lentos— es común.

4.5 Relevancia

La relevancia es doble. *Hacia atrás*: el aparato cuántico (modos del generador, solapamientos, diagnósticos universales) proporciona al efecto macroscópico clásico el fundamento teórico riguroso que históricamente le faltó, convirtiendo una curiosidad sobre el agua en un teorema sobre la relajación fuera del equilibrio. *Hacia adelante*: muestra que el mismo fenómeno, y las mismas herramientas computacionales (descomposición espectral, evolución del operador densidad, trayectorias), abarcan sin costura desde el vaso de agua hasta el procesador cuántico, lo que justifica tratar el efecto Mpemba como un *principio general de diseño* de protocolos de relajación acelerada.